

(BFS)

```
graph = {  
    '5': ['3', '7'],  
    '3': ['2', '4'],  
    '7': ['8'],  
    '2': [],  
    '4': [],  
    '8': []  
}  
visited = []  
queue = ['5']  
while queue:  
    node = queue.pop(0)  
    if node not in visited:  
        print(node, end=" ")  
        visited.append(node)  
        queue.extend(graph[node])
```

(DFS)

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F', 'G'],  
    'D': [],  
    'E': [],  
    'F': [],  
    'G': []  
}  
visited = []  
def dfs(node):  
    if node not in visited:  
        print(node, end=" ")  
        visited.append(node)  
        for i in graph[node]:  
            dfs(i)  
print("Depth First Search:")  
dfs('A')
```

(WATER JUG)

from collections import deque

q = deque([(0, 0)])

visited = {(0, 0)}

while q:

a, b = q.popleft()

print((a, b))

if a == 2:

print("Goal State Reached")

break

states = [

(4, b), (a, 3), (0, b), (a, 0),

(a - min(a, 3 - b), b + min(a, 3 - b)),

(a + min(b, 4 - a), b - min(b, 4 - a))

]

for s in states:

if s not in visited:

visited.add(s)

q.append(s)

(HILL)

import random

def hill(arr):

c = random.randint(0, len(arr)-1)

while True:

if c > 0 and arr[c-1] > arr[c]:

c -= 1

elif c < len(arr)-1 and arr[c+1] > arr[c]:

c += 1

else:

break

return c, arr[c]

data = [10, 20, 5, 23, 90, 67]

i, v = hill(data)

print("Peak Index:", i)

print("Peak Value:", v)

(MAGIC)AI

m = [[8,1,6],

[3,5,7],

[4,9,2]]

if (sum(m[0])==15 and sum(m[1])==15 and sum(m[2])==15 and

m[0][0]+m[1][0]+m[2][0]==15 and

m[0][1]+m[1][1]+m[2][1]==15 and

m[0][2]+m[1][2]+m[2][2]==15 and

m[0][0]+m[1][1]+m[2][2]==15 and

m[0][2]+m[1][1]+m[2][0]==15):

print("Valid Magic Square")

else:

print("Invalid Magic Square")

(MAX)

```
def find_maximum(data):
```

```
    max_value = data[0]
```

```
    for value in data:
```

```
        if value > max_value:
```

```
            max_value = value
```

```
    return max_value # Placed outside the loop
```

```
numbers = [10, 25, 7, 77, 18]
```

```
result = find_maximum(numbers)
```

```
print("Maximum value:", result)
```

(LOGIC EX)

p = True

q = False

exp = input("Enter (&,|,!,>): ")

if exp == "&":

print("Result:", p and q)

elif exp == "|":

print("Result:", p or q)

elif exp == "!":

print("Result:", not p)

elif exp == ">":

print("Result:", (not p) or q)

else:

print("Invalid Expression")

(WEATHER)

def weather(temp, hum):

if temp > 35:

return "Hot Weather"

elif temp < 20:

return "Cold Weather"

elif hum > 80:

return "Rainy Weather"

else:

return "Pleasant Weather"

temp = int(input("Enter Temperature: "))

hum = int(input("Enter Humidity: "))

print("Weather:", weather(temp, hum))

(AI AGENT)

```
def simple_ai_agent(room_status):  
    if room_status == "dirty":  
        return "Action: Clean the room"  
    else:  
        return "Action: Do nothing"  
print("Simple AI Agent")  
room = input("Enter room status (clean/dirty): ").strip().lower()  
decision = simple_ai_agent(room)  
print(decision)
```

(CHAT BOT)

while True:

msg = input("You: ").lower()

if msg == "hi":

print("Bot: Hello!")

elif msg == "how are you":

print("Bot: I am fine.")

elif msg == "bye":

print("Bot: Goodbye!")

break

else:

print("Bot: I don't understand.")